

Dossier d'Architecture Technique (DAT) - SISS

Table des Matières

- 1. Introduction
- 2. Contexte et Périmètre
- 3. Architecture Fonctionnelle
- 3.1 Cas d'Usage - 3.2 Règles Métier
- 4. Architecture Technique
- 4.1 Architecture Logique - 4.2 Architecture Physique - 4.3 Composants Techniques - 4.4 Flux de Données
- 5. Diagrammes Techniques
- 5.1 Diagramme de Déploiement - 5.2 Diagramme de Composants - 5.3 Diagramme de Séquence - 5.4 Diagramme d'États
- 5.5 Diagramme de Classes
- 6. Configuration et Déploiement
- 7. Sécurité
- 8. Annexes

1. Introduction

Le **SISS** (Système d'Information pour le Suivi des Stockages) est une application conçue pour gérer l'archivage physique et numérique de documents sensibles dans un contexte administratif ministériel. Elle assure la traçabilité des mouvements, la gestion sécurisée des accès et le stockage conforme des pièces jointes.

2. Contexte et Périmètre

2.1 Description Générale

- **ID** : 583
- **Statut** : En construction

- **Numéro d'affaire** : AFF0583
- **Acteur MOE** : SG/DNUM/PNM/DPNM3
- **Technologie principale** : Java

2.2 Domaine Applicatif

L'application **SISS** gère :

- L'archivage physique de documents sensibles.
- La traçabilité des mouvements.
- La gestion des accès sécurisés.
- Le stockage des pièces jointes.

2.3 Périmètre Fonctionnel

- **Inclus** :

- Versements de documents. - Demandes d'accès, consultation et restitution. - Mouvements de documents (transferts, archivage, désarchivage). - Gestion des référentiels (utilisateurs, départements) via fichiers CSV. - Authentification (SAML2 ou mode TEST local). - Stockage sécurisé des pièces jointes (AWS S3-compatible). - Envoi d'e-mails via API HEDWIGE ou SMTP. - Batches automatisés (fin de délégation, nettoyage).

- **Exclus** :

- Gestion de données patients. - Logique de facturation ou tarification. - Workflow avancé (pas de moteur BPM).

3. Architecture Fonctionnelle

3.1 Cas d'Usage

```
@startuml
title Cas d'usage
left to right direction
actor "Utilisateur" as User
rectangle "SISS" {
    User -- (Gérer versements)
    User -- (Consulter dossiers)
    User -- (Demander restitution)
    User -- (Recevoir notifications)
    (Gérer versements) .> (Stockage PJ) : inclut
    (Recevoir notifications) .> (Envoyer email) : inclut
}
@enduml
```

Fig 3.1 Cas d'usage

3.2 Règles Métier

- **Gestion des pièces jointes** :

- Formats autorisés : Configurables via `sis.format_piece_jointe` (ex. : `pdf`, `docx`). - Stockage : AWS S3-compatible (`sis.pj-stockage.mode=AWS`). - Chemin temporaire local : `/app/data/sis_upload` .
- **Chiffrement des données :**
- Clé symétrique définie dans `sis.encryption.key` . - Non modifiable après initialisation. - Générée via `sis-generate-key.jar` .

4. Architecture Technique

4.1 Architecture Logique

Composant	Technologie	Description
Backend	Java Spring Boot	Application embarquée dans **Tomcat 9.0.82** (JDK 11).
Frontend	Inclus dans le <code>`war`</code>	Pas de séparation explicite avec le backend.
Base de données	PostgreSQL 13.19	Stockage des données métiers et des référentiels.
Stockage des PJ	AWS S3-compatible	Stockage sécurisé des pièces jointes.
Authentification	SAML2 / Stub LDAP	Authentification centralisée ou locale (mode TEST).
Envoi d'e-mails	API HEDWIGE / SMTP	Intégration avec le service ministériel d'e-mails sécurisés.

4.2 Architecture Physique

- **Environnements :**
- **Développement** : Sources locales (fichiers `.tar.bz2` copiés dans l'image Docker). - **Production** : Sources téléchargées depuis **Nexus Silicom** (`nexus.sro.silicom.fr`).

4.3 Composants Techniques

```
@startuml
title Composants Techniques
package "SISS Application" {
    [Tomcat 9 + SISS.war] as app
    [PostgreSQL 13] as db
    [AWS S3] as storage
    [HEDWIGE API] as mail
}

app --> db : JDBC
app --> storage : S3 API
app --> mail : HTTPS + OAuth2
@enduml
```

Fig 4.3 Composants Techniques

4.4 Flux de Données

```
@startuml
title Flux de Données
actor Utilisateur
Utilisateur -> ((Tomcat SISS2.war)) : Requetes
((Tomcat SISS.war)) -> (PostgreSQL 13) : JDBC
((Tomcat SISS.war)) -> (AWS S3) : S3 API
((Tomcat SISS.war)) -> (HEDWIGE API) : HTTPS + OAuth2
@enduml
```

Fig 4.4 Flux de Données

5. Diagrammes Techniques

5.1 Diagramme de Déploiement

```
@startuml
title Diagramme de Déploiement
node "Host Dev" {
    node "Docker Engine" {
        artifact "siss-app" as app
        artifact "siss-db" as db
    }
}
app --> db : localhost:5432
app ..> "Local FS" : ./data, ./logs
@enduml
```

Fig 5.1 : Diagramme de Déploiement

5.2 Diagramme de Composants

```
@startuml
title Diagramme de Composants
package "SISS Application" {
    [Tomcat 9 + SISS.war] as app
    [PostgreSQL 13] as db
    [AWS S3] as storage
    [HEDWIGE API] as mail
}

app --> db : JDBC
app --> storage : S3 API
app --> mail : HTTPS + OAuth2
@enduml
```

Fig 5.2 Diagramme de Composants

5.3 Diagramme de Séquence : Authentification en Mode TEST

```

@startuml
title Authentification en Mode TEST
actor Utilisateur
participant "SISS App" as App
database "PostgreSQL" as DB

Utilisateur -> App : POST /login (user, pwd)
App -> DB : SELECT * FROM ref_utilisateur WHERE uti_login = ?
DB --> App : Utilisateur
App -> App : Valider contre authentication.stubLdap.[user].*
App --> Utilisateur : Session créée
@enduml

```

Fig 5.3 : Authentification en Mode TEST

5.4 Diagramme d'États : Cycle de Vie d'un Utilisateur

```

@startuml
title Cycle de Vie d'un Utilisateur
[*] --> Inactif
Inactif --> Actif : Initialisation CSV
Actif --> Délégue : Délégation active
Délégue --> Actif : Fin délégation (batch)
Actif --> [*] : Désactivation manuelle
@enduml

```

Fig 5.4 : Cycle de Vie d'un Utilisateur

5.5 Diagramme de Classes : Modèle Relationnel Simplifié

```

@startuml
title Modèle Relationnel Simplifié
class ref_utilisateur {
    + String uti_login
    + String cn
    + String mail
    + String departement
}

class dossier {
    + Long id
    + String statut
    + Date date_creation
}

ref_utilisateur "1" -- "0..*" dossier : crée
@enduml

```

Fig 5.5 : Modèle Relationnel Simplifié

6. Configuration et Déploiement

6.1 Paramétrage

- **Fichiers de configuration :**

- `application.properties` : Paramètres principaux. - `access.properties` : Accès sécurisés. - `ref_utilisateur.csv` : Initialisation des utilisateurs.

- **Clé de chiffrement** :

- Générée via la commande :

```
docker exec -it siss:latest java -jar /app/siss/lib/version/siss-generate-key.jar [cle-de-chiffrement
```

- Résultat à renseigner dans `siss.encryption.key` .

6.2 Déploiement

- **Build locale** :

```
docker build -t siss:latest --build-arg SISS_VERSION=X.Y.Z -f conf/dev/Dockerfile .
```

- **Build CI/CD** : - Image disponible sous `eu.gcr.io/dpnm3-lab/siss` .

7. Sécurité

- **Chiffrement des données** :

- Clé symétrique pour le chiffrement/déchiffrement des données. - Non modifiable après initialisation.

- **Authentification** :

- SAML2 pour la production. - Stub LDAP pour les tests.

8. Annexes

- **Références** :

- Ce document suit les principes de **arc42** pour une documentation claire et orientée décision. - Diagrammes centrés sur les besoins.

[↩ Retour au sommaire](#)

Points clés du DAT :

1. **Diagrammes PlantUML** : - Tous les diagrammes sont intégrés directement dans le document. - Ils couvrent les aspects fonctionnels et techniques (cas d'usage, composants, séquences, états, classes).

2. **Structure claire** : - Sections bien définies pour chaque aspect de l'architecture. - Navigation facile grâce aux liens de retour vers la TOC.

3. **Conformité** : - Respect des règles de **PlantUML** et des exigences techniques identifiées dans les documents sources.